

# Clean Code

## Tips & Tricks

11 categories · 38 safe rules · before & after examples

This guide covers the safest, highest-impact refactoring rules for Java code. Every rule can be applied immediately — even by juniors — without risk of breaking existing behavior. AI tools and static analysers (SonarQube, IntelliJ) can detect and apply most of these automatically.

Each section shows **BEFORE** (problematic code) and **AFTER** (improved code) with a short explanation of **WHY** the change matters.

| #  | Category     | Key Benefit                   | Detected By          |
|----|--------------|-------------------------------|----------------------|
| 1  | Naming       | Code reads like prose         | AI, IntelliJ         |
| 2  | Logic        | Eliminates nesting            | IntelliJ, SonarQube  |
| 3  | Structure    | Removes dead weight           | SonarQube, IntelliJ  |
| 4  | Null Safety  | Prevents NPE at compile time  | IntelliJ, SonarQube  |
| 5  | Immutability | Signals intent, prevents bugs | IntelliJ, Checkstyle |
| 6  | Formatting   | Consistent, scannable code    | Checkstyle, IDE      |
| 7  | Exceptions   | No silent failures            | SonarQube            |
| 8  | Collections  | Modern, expressive APIs       | IntelliJ, SonarQube  |
| 9  | Comments     | Signal-to-noise ratio up      | AI, code review      |
| 10 | Boilerplate  | Compiler-verified overrides   | IntelliJ, compiler   |
| 11 | Tests        | Cleaner failure messages      | IntelliJ             |

## 1 Naming

Good names are the single highest-ROI investment in code quality. A well-named variable or method eliminates the need for comments and makes code read like plain English. Aim for names that reveal intent, not implementation.

### Expand abbreviations

**WHY:** Abbreviations force readers to decode meaning. Full names are self-explanatory.

✗ BEFORE

```
String usr = getUsr();
int d = 30;
Config cfg = load();
```

✓ AFTER

```
String user = getUser();
int deliveryDays = 30;
Config config = load();
```

### Name booleans as questions

**WHY:** Boolean names that read as questions make conditions read naturally.

✗ BEFORE

```
boolean active = true;
if (valid) { ... }
boolean flag = check();
```

✓ AFTER

```
boolean isActive = true;
if (isValid) { ... }
boolean isEligible = check();
```

### Remove redundant context

**WHY:** Class name already provides context. Repeating it in fields adds noise.

✗ BEFORE

```
class User {
    String userName;
    int userAge;
}
```

✓ AFTER

```
class User {
    String name;
    int age;
}
```

## Constants in SCREAMING\_SNAKE\_CASE

**WHY:** Convention instantly signals: this is a constant, it never changes.

✗ BEFORE

```
static final int secondsInHour = 3600;
static final String defaultRole = "USER";
```

✓ AFTER

```
static final int SECONDS_IN_HOUR = 3600;
static final String DEFAULT_ROLE = "USER";
```

## 2 Logic Simplification

Complex conditions and nested if-blocks are the leading cause of bugs and the hardest code to review. Flattening logic and naming conditions makes the code's intent obvious at a glance.

### Simplify boolean returns

**WHY:** The if/else is redundant — the condition IS the result.

✗ BEFORE

```
if (age >= 18) {
    return true;
} else {
    return false;
}
```

✓ AFTER

```
return age >= 18;
```

### Remove x == true / x == false

**WHY:** Comparing a boolean to true or false is circular and noisy.

✗ BEFORE

```
if (isActive == true) { ... }
if (isValid == false) { ... }
```

✓ AFTER

```
if (isActive) { ... }
if (!isValid) { ... }
```

## Extract complex conditions

**WHY:**Named booleans document intent. The if statement becomes a sentence.

✗ BEFORE

```
if (user.getAge() >= 18
    && user.isVerified()
    && !user.isBanned()) {
```

✓ AFTER

```
boolean canPurchase = user.getAge() >= 18
    && user.isVerified()
    && !user.isBanned();
if (canPurchase) {
```

## Guard clauses (early return)

**WHY:**Eliminate nesting. Happy path flows uninterrupted at the left margin.

✗ BEFORE

```
if (user != null) {
    if (user.isActive()) {
        process(user);
    }
}
```

✓ AFTER

```
if (user == null) return;
if (!user.isActive()) return;
process(user);
```

## 3 Code Structure

Clean structure means a reader can scan a class and understand its shape in seconds. Magic numbers, dead code, and System.out calls are symptoms of code that was never tidied after it started working.

## Replace magic numbers with constants

**WHY:**3600 means nothing alone. SECONDS\_IN\_HOUR tells a story.

✗ BEFORE

```
if (elapsed > 3600) {
    expire(session);
}
```

✓ AFTER

```
static final int SECONDS_IN_HOUR = 3600;

if (elapsed > SECONDS_IN_HOUR) {
    expire(session);
}
```

## Remove System.out.println

**WHY:** Debug prints leak into production and flood logs with noise. Use SLF4J.

✗ BEFORE

```
System.out.println("Processing: " + id);
System.out.println("Done");
```

✓ AFTER

```
log.debug("Processing order: {}", id);
log.info("Order processing complete");
```

## Convert lambda to method reference

**WHY:** Method references are shorter and eliminate the intermediate variable.

✗ BEFORE

```
list.stream()
    .map(s -> s.toUpperCase())
    .collect(toList());
```

✓ AFTER

```
list.stream()
    .map(String::toUpperCase)
    .collect(toList());
```

## Remove dead code

**WHY:** Commented-out code creates confusion. Git remembers history — delete it.

✗ BEFORE

```
// String old = compute();
// if (old != null) use(old);
String result = computeNew();
```

✓ AFTER

```
String result = computeNew();
```

4

## Null Safety

`NullPointerException` is still the most common exception in Java. Annotations, empty collections, and fail-fast validation push null problems to the surface immediately rather than hiding them deep in logic.

## Annotate with @NotNull / @Nullable

**WHY:** Annotations are a contract. IDEs and tools warn callers before runtime.

✗ BEFORE

```
public String findName(Long id) {  
    return repo.find(id);  
}
```

✓ AFTER

```
public @Nullable String findName(@NotNull Long id) {  
    return repo.find(id);  
}
```

## Return empty collection, not null

**WHY:** Callers can iterate an empty list safely. Null forces a null-check.

✗ BEFORE

```
public List<Order> getOrders() {  
    if (orders == null) return null;  
    return orders;  
}
```

✓ AFTER

```
public List<Order> getOrders() {  
    if (orders == null) return List.of();  
    return orders;  
}
```

## Fail fast with requireNonNull

**WHY:** Validate at entry. The error message pinpoints exactly what is null.

✗ BEFORE

```
public void process(Order order) {  
    // order might be null...  
    order.calculate();  
}
```

✓ AFTER

```
public void process(@NotNull Order order) {  
    Objects.requireNonNull(order, "order must not be null");  
    order.calculate();  
}
```

## Use requireNonNullElse for defaults

**WHY:** Cleaner than a ternary. Reads as: use this, or fall back to default.

✗ BEFORE

```
String name = user != null ? user : "Anonymous";
```

✓ AFTER

```
String name = Objects.requireNonNullElse(user, "Anonymous");
```

## 5 Immutability

Immutability eliminates an entire class of bugs: accidental reassignment. The `final` keyword costs nothing at runtime but communicates intent clearly: this value does not change from this point forward.

### Add final to local variables

**WHY:** Readers don't have to track whether the variable changes later.

✗ BEFORE

```
String status = fetchStatus();
int count = list.size();
return process(status, count);
```

✓ AFTER

```
final String status = fetchStatus();
final int count = list.size();
return process(status, count);
```

### Add final to method parameters

**WHY:** Prevents accidental reassignment of the parameter inside the method.

✗ BEFORE

```
public void send(String message, User user) {
    message = message.trim(); // reassigning param!
    user.notify(message);
}
```

✓ AFTER

```
public void send(final String message, final User user) {
    final String trimmed = message.trim();
    user.notify(trimmed);
}
```

### Prefer primitive over wrapper type

**WHY:** `boolean` can't be null. `Boolean` can — hiding a potential NPE.

✗ BEFORE

```
private Boolean isActive;
private Integer retryCount = 0;
```

✓ AFTER

```
private boolean isActive;
private int retryCount = 0;
```

## 6 Formatting & Style

Consistent formatting lets readers focus on logic, not layout. Blank lines are paragraph breaks — use them to group validation, business logic, and return statements into readable sections.

## Group code into logical blocks

**WHY:** Blank lines guide the eye. Three clear sections replace the need for comments.

✗ BEFORE

```
public Order create(String id, int qty) {
    if (id == null) throw new IllegalArgumentException();
    int price = catalog.getPrice(id);
    int total = price * qty;
    return new Order(id, qty, total);
}
```

✓ AFTER

```
public Order create(String id, int qty) {
    if (id == null) throw new IllegalArgumentException();

    int price = catalog.getPrice(id);
    int total = price * qty;

    return new Order(id, qty, total);
}
```

## Unify logger style (SLF4J everywhere)

**WHY:** Mixed logging frameworks make log aggregation and config painful.

✗ BEFORE

```
System.out.println("Error: " + e);
Logger log2 = Logger.getLogger(Cls.class.getName());
log2.info("done");
```

✓ AFTER

```
private static final Logger log =
    LoggerFactory.getLogger(OrderService.class);

log.error("Error processing order", e);
log.info("Order complete");
```

## 7

## Exceptions

Empty catch blocks are silent killers — the application continues in a broken state with no trace of what went wrong. Always log or rethrow. Never swallow.



### Never leave catch blocks empty

**WHY:** An empty catch hides failures. You lose all diagnostic information.

✗ BEFORE

```
try {
    order.process();
} catch (Exception e) {
    // TODO fix later
}
```

✓ AFTER

```
try {
    order.process();
} catch (Exception e) {
    log.warn("Order processing failed: {}", e.getMessage(), e);
}
```

### Remove unnecessary casts

**WHY:** Casting an already-typed variable adds noise and can hide errors.

✗ BEFORE

```
String name = (String) user.getName();
List<String> items = (List<String>) getItems();
```

✓ AFTER

```
String name = user.getName();
List<String> items = getItems();
```

## 8 Collections & Loops

Modern Java gives you expressive, safe collection APIs. Using `isEmpty()` over `size()==0`, for-each over indexed loops, and the diamond operator are small changes with immediate readability gains.

### Use `isEmpty()` instead of `size() == 0`

**WHY:** `isEmpty()` reads as a question. `size()==0` is an arithmetic check.

✗ BEFORE

```
if (list.size() == 0) { ... }
if (list.size() > 0) { ... }
```

✓ AFTER

```
if (list.isEmpty()) { ... }
if (!list.isEmpty()) { ... }
```

### Use for-each when index is unused

**WHY:** for-each removes boilerplate and eliminates off-by-one errors.

✗ BEFORE

```
for (int i = 0; i < orders.size(); i++) {  
    process(orders.get(i));  
}
```

✓ AFTER

```
for (Order order : orders) {  
    process(order);  
}
```

### Add diamond operator

**WHY:** The compiler infers the type. Repeating it is redundant noise.

✗ BEFORE

```
List<String> names = new ArrayList<String>();  
Map<String, Integer> map = new HashMap<String, Integer>();
```

✓ AFTER

```
List<String> names = new ArrayList<>();  
Map<String, Integer> map = new HashMap<>();
```

### Use list.sort() over Collections.sort()

**WHY:** Instance method is more readable and follows modern Java style.

✗ BEFORE

```
Collections.sort(orders);  
Collections.sort(orders, comparator);
```

✓ AFTER

```
orders.sort(null);  
orders.sort(comparator);
```

9

## Comments

A comment that explains WHAT code does is a code smell — it means the code isn't clear enough on its own. Good comments explain WHY: business rules, constraints, workarounds, trade-offs.

## Delete what-comments

**WHY:** Code already shows what it does. The comment just duplicates it.

✗ BEFORE

```
// increment counter
count++;

// call the service
service.execute(order);
```

✓ AFTER

```
count++;
service.execute(order);
```

## Remove commented-out code

**WHY:** Commented code misleads readers. Git has the history — delete it.

✗ BEFORE

```
// String old = legacyCompute();
// result = transform(old);
String result = computeNew();
```

✓ AFTER

```
String result = computeNew();
```

10

## Annotations & Boilerplate

Missing `@Override` is a silent trap. If you rename or remove a parent method, the compiler won't warn you — your override becomes an unrelated new method. One annotation prevents a whole class of bugs.

## Always add `@Override`

**WHY:** Compiler validates the override. Rename the parent — compiler tells you immediately.

✗ BEFORE

```
public class PremiumOrder extends Order {
    public BigDecimal calculateTotal() {
        return super.calculateTotal().multiply(DISCOUNT);
    }
}
```

✓ AFTER

```
public class PremiumOrder extends Order {
    @Override
    public BigDecimal calculateTotal() {
        return super.calculateTotal().multiply(DISCOUNT);
    }
}
```

## 11 Test Assertions

Clear test assertions make failure messages self-explanatory. `assertTrue(x)` reads as a sentence. `assertEquals(true, x)` forces mental parsing and produces worse failure output.

### Use `assertTrue` / `assertFalse` / `assertNull`

**WHY:** Semantic assertions produce cleaner failure messages and read naturally.

#### ✗ BEFORE

```
assertEquals(true, user.isActive());
assertEquals(false, list.isEmpty());
assertEquals(null, result);
```

#### ✓ AFTER

```
assertTrue(user.isActive());
assertFalse(list.isEmpty());
assertNull(result);
```



## How to Use This Guide

### 1 Start with EDUCATIONAL mode

Run the AI prompt in EDUCATIONAL mode first. It shows what it would change without touching any code. Review the findings — this is the fastest way to train your eye for code quality issues.

### 2 Apply SAFE rules first

Use the SAFE prompts to apply the unambiguous changes: isEmpty(), @Override, final on locals, removing System.out. Review the git diff — every change should be obvious and risk-free.

### 3 Use AGGRESSIVE for bigger gains

Once comfortable, run the AGGRESSIVE variant. It will rename variables, extract conditions, reorder class members. Review carefully — some suggestions may need human judgment to finalize the naming.

### 4 Enable SonarQube + IntelliJ inspections

These tools catch most of the rules in this guide automatically. Turn on all quality inspections in IntelliJ and address the highlights. SonarQube can enforce the same rules in your CI pipeline.

### 5 Review AI output before committing

AI refactoring is fast but not perfect. Always review the diff in git before committing. Pay extra attention to: renamed variables (correct domain term?), extracted conditions (does the name express business intent?).

### 6 Rules that need human judgment

Some changes are NOT in this guide because they require context: @EqualsAndHashCode (dangerous with JPA), @Builder (breaks API), Optional return types (breaks callers), Stream conversion (changes stack traces). These are Level 2 topics.

[improvecode.ai](https://improvecode.ai) · Full course, interactive examples, and AI prompts available at [improvecode.ai](https://improvecode.ai)